



TẬP ĐOÀN BƯU CHÍNH VIỄN THÔNG VIỆT NAM
VNPT IT - TRUNG TÂM DMS
Bộ phận Phát triển Giải pháp AI

BÁO CÁO KIỂM THỬ HIỆU NĂNG HỆ THỐNG HỎI ĐÁP DỮ LIỆU TEXT-TO-SQL DƯỚI TẢI NGƯỜI DÙNG ĐỒNG THỜI

Load Test bằng công cụ k6 — Xác định ngưỡng bão hoà
trên nền tảng Vanna, Ollama 1lama3.2 và PostgreSQL Neon

PHẠM VI KIỂM THỬ

Báo cáo trình bày kết quả kiểm thử tải (load test) endpoint hỏi đáp AI /api/vanna/v2/chat_sse của hệ thống PoC Text-to-SQL. Mục tiêu là mô phỏng số lượng người dùng đồng thời tăng dần (5, 15, 30 và 50 người dùng), đo thời gian phản hồi, thông lượng và tỉ lệ lỗi, từ đó xác định điểm bão hoà và đề xuất phương án nâng cao khả năng chịu tải. Báo cáo cũng ghi nhận **đợt tối ưu tầng ứng dụng** được thực hiện sau kiểm thử lần đầu và kết quả đo lại ở cùng mức tải 50 người dùng.

Tóm tắt thông số kiểm thử

Công cụ kiểm thử:	k6 v2.1.0, kịch bản <code>ramping-vus</code>
Đối tượng:	Endpoint SSE của Vanna 2.0 (FastAPI, Unicorn 1 worker)
Hạ tầng AI:	Ollama 1lama3.2 cục bộ, PostgreSQL trên Neon
Mức tải:	5 → 15 → 30 người dùng và cao điểm 50 người dùng
Kết quả trước tối ưu:	50 user: tỉ lệ lỗi 13%, p95 = 139 giây
Kết quả sau tối ưu:	50 user: tỉ lệ lỗi 0%, p95 = 47,0 giây, thông lượng +196%
Nút thắt còn lại:	Tốc độ sinh token của LLM trên CPU (cần gấp ≈ 5 lần)

Người thực hiện:
Lê Đức Anh
Trung tâm DMS

Mục lục

1	Tóm tắt kết quả cho Lãnh đạo (Executive Summary)	2
2	Môi trường và phương pháp kiểm thử	2
3	Kết quả kiểm thử	5
4	Phân tích kết quả	7
5	Tối ưu hoá tầng ứng dụng và kết quả đo lại	8
6	Thử nghiệm cấu hình tầng suy luận	10
7	Đánh giá theo tiêu chí nghiệm thu (SLA)	12
8	Giới hạn của kiểm thử	12
9	Kết luận	13
10	Khuyến nghị cải thiện	13
	Phụ lục A. Kịch bản k6	16
	Tài liệu tham khảo	17

1. Tóm tắt kết quả cho Lãnh đạo (Executive Summary)

Báo cáo đánh giá khả năng chịu tải của hệ thống hỏi đáp dữ liệu bằng ngôn ngữ tự nhiên (Text-to-SQL) khi số lượng người dùng truy cập đồng thời tăng cao. Công cụ **k6** được dùng để phát sinh tải thực tế lên endpoint streaming `/api/vanna/v2/chat_sse`, mô phỏng các câu hỏi nghiệp vụ tiếng Việt.

Kiểm thử được thực hiện qua **hai đợt**. Đợt đầu đo hiện trạng: hệ thống vận hành ổn định dưới 10 người dùng đồng thời, nhưng khi vượt ngưỡng này thì bão hoà — thông lượng không tăng thêm mà độ trễ và tỉ lệ lỗi tăng phi tuyến. Ở mức 50 người dùng, độ trễ p95 đạt **139 giây** và tỉ lệ lỗi lên tới **13%**.

Phân tích mã nguồn sau đợt đầu phát hiện nguyên nhân **không nằm ở câu lệnh SQL hay ở cơ sở dữ liệu**, mà ở cách tầng ứng dụng gọi chúng: thư viện truy cập PostgreSQL và thư viện gọi Ollama đều là thư viện **đồng bộ** nhưng được gọi trực tiếp trong vòng lặp sự kiện bất đồng bộ (asyncio) của máy chủ. Hệ quả là mỗi lần chờ cơ sở dữ liệu hoặc chờ mô hình sinh chữ, **toàn bộ tiến trình bị đóng băng** — 50 người dùng đồng thời trên thực tế bị phục vụ nối đuôi nhau. Ngoài ra mỗi truy vấn còn mở một kết nối mới tới Neon, tốn một lượt bắt tay TLS xuyên quốc gia trước khi chạy được câu lệnh.

Sau khi khắc phục (Mục 5) và phát hiện thêm rằng `OLLAMA_NUM_PARALLEL` để mặc định bằng 1 — tức máy chủ mô hình xử lý tuần tự — nâng tham số này lên 4 (Mục 6), kiểm thử lại ở đúng mức 50 người dùng cho kết quả: tỉ lệ thành công **100%** (từ 87%), thông lượng **tăng 196%** lên 2,07 req/giây và p95 **giảm 66%** xuống 47,0 giây.

Kết luận nhanh

Đợt tối ưu tầng ứng dụng đã **loại bỏ hoàn toàn lỗi** ở mức 50 người dùng và nâng thông lượng từ 0,70 lên 1,10 request/giây, nhưng độ trễ tuyệt đối vẫn cao. Nút thắt còn lại là **tốc độ suy luận của mô hình ngôn ngữ trên CPU**: đo trực tiếp cho thấy llama3.2 sinh khoảng **18 token/giây** do máy không có GPU rời. Đây là giới hạn phần cứng, không thể gỡ tiếp bằng tối ưu mã nguồn; muốn giảm độ trễ xuống dưới 10 giây cần GPU rời, mô hình nhỏ hơn, hoặc chuyển sang API LLM đám mây [3, 2].

2. Môi trường và phương pháp kiểm thử

2.1. Cấu hình môi trường

Bảng 1: Cấu hình môi trường kiểm thử

Hạng mục	Chi tiết
Công cụ kiểm thử	k6 v2.1.0 (Windows/amd64)
Endpoint	POST /api/vanna/v2/chat_sse (Server-Sent Events)
Mô hình AI	Ollama llama3.2 (chạy cục bộ, không tốn phí API)
Cơ sở dữ liệu	PostgreSQL — Neon, schema qlsp_backup
Máy chủ ứng dụng	Uvicorn (1 worker), FastAPI, Vanna 2.0
Loại truy vấn	Câu hỏi tiếng Việt (đếm và tổng hợp đơn giản)
Tiêu chí thành công	HTTP 200 và stream kết thúc bằng [DONE]
Timeout mỗi request	180 giây
Baseline (1 người dùng)	≈ 2,5 giây/truy vấn (trước tối ưu)
Baseline sau tối ưu	≈ 0,1 giây (câu chỉ chạm CSDL) — 5,5 giây (câu gọi LLM)

2.2. Cấu hình phần cứng

Toàn bộ hệ thống (ứng dụng, mô hình Ollama và công cụ k6) chạy trên cùng một máy trạm; cơ sở dữ liệu PostgreSQL đặt từ xa trên Neon (cloud). Đây là mô hình kiểm thử một máy (single-node), phù hợp cho giai đoạn PoC.

Bảng 2: Thông số phần cứng máy chạy kiểm thử

Thành phần	Thông số
CPU	AMD Ryzen 7 8845H (8 nhân / 16 luồng)
GPU	AMD Radeon 780M (đồ hoạ tích hợp)
RAM	16 GB (khoảng 13,8 GB khả dụng; phần còn lại chia sẻ cho GPU tích hợp)
Hệ điều hành	Windows 11 Home
Vị trí cơ sở dữ liệu	Neon (PostgreSQL đám mây, kết nối qua SSL)

Ghi chú: Do máy sử dụng **GPU tích hợp** (không có GPU rời), mô hình llama3.2 suy luận hoàn toàn trên CPU. Lệnh `ollama ps` trong lúc kiểm thử xác nhận mô hình chạy ở chế độ 100% CPU. Đây là yếu tố phần cứng trực tiếp giới hạn tốc độ và mức độ song song của mô hình, và cần được tính đến khi diễn giải kết quả kiểm thử.

2.3. Đặc điểm kỹ thuật ảnh hưởng đến hiệu năng

- Mỗi request sử dụng một `conversation_id` riêng biệt để tránh cơ chế khoá theo hội thoại (HTTP 409) của máy chủ, bảo đảm tải thực sự song song.

- Trong 6 câu hỏi kiểm thử, **5 câu đi theo nhánh sinh SQL tắt định** của `main.py` (hệ thống nhận dạng mẫu câu đếm và dựng thẳng câu lệnh SQL, **không gọi mô hình**). Chỉ câu “Liệt kê 5 đơn vị đầu tiên theo tên” thực sự gọi `llama3.2`. Vì vậy chi phí LLM tập trung vào $\approx 1/6$ lượng request, nhưng đủ để chiếm trọn CPU và kéo chậm toàn bộ phần còn lại.
- Ollama cục bộ suy luận trên CPU nên là điểm giới hạn thông lượng chính của hệ thống. Đo trực tiếp qua API `/api/chat`: nạp prompt hệ thống (1.202 token) chỉ mất 0,28 giây, nhưng **sinh 118 token mất 6,53 giây**, tương đương ≈ 18 token/giây.
- Máy chủ FastAPI chạy bất đồng bộ (asyncio) nhưng **trước tối ưu** lại gọi thư viện đồng bộ (`psycpg2`, `ollama`) ngay trong vòng lặp sự kiện, khiến các request không thực sự chạy song song. Chi tiết ở Mục 4.

2.4. Kịch bản kiểm thử

Hai kịch bản tăng tải (`ramping-vus`) được thực hiện:

1. **Kịch bản A — Tăng dần:** 5 $\rightarrow$ 15 $\rightarrow$ 30 người dùng đồng thời, tổng thời gian $\approx$ 3 phút 35 giây. Mục tiêu quan sát diễn biến khi tải tăng từ thấp đến trung bình.
2. **Kịch bản B — Cao điểm 50:** tăng nhanh lên 50 người dùng và giữ trong 120 giây. Mục tiêu kiểm tra hành vi khi vượt xa ngưỡng bảo hoà.

Listing 1: Trích cấu hình kịch bản tăng tải trong `k6`

```
export const options = {
  scenarios: {
    ramping_users: {
      executor: 'ramping-vus',
      stages: [
        { duration: '20s', target: 5 }, // 5 người dùng
        { duration: '20s', target: 15 }, // tăng lên 15
        { duration: '60s', target: 30 }, // cao điểm 30
      ],
    },
  },
};
```

2.5. Bộ câu hỏi kiểm thử

Mỗi người dùng ảo chọn ngẫu nhiên một câu hỏi trong tập câu hỏi nghiệp vụ tiếng Việt dưới đây. Các câu hỏi thuộc nhóm đếm và tổng hợp đơn giản nhằm cô lập chi phí suy luận của mô hình (tách khỏi chi phí truy vấn SQL phức tạp):

1. “Có bao nhiêu người dùng trong hệ thống?”
2. “Có bao nhiêu đơn vị trong bảng `units`?”
3. “Có bao nhiêu `spdv_tickets`?”
4. “Đếm số người dùng theo trạng thái.”
5. “Liệt kê 5 đơn vị đầu tiên theo tên.”

6. “Có bao nhiêu sự kiện trong `spdv_ticket_events`?”

3. Kết quả kiểm thử

Mục này trình bày kết quả **đợt kiểm thử thứ nhất** (đo hiện trạng). Kết quả đo lại sau khi tối ưu được trình bày ở Mục 5.

3.1. Kết quả tổng quan

Bảng 3: So sánh kết quả tổng quan theo mức tải

Chỉ số	1 user (baseline)	30 user (A)	50 user (B)
Tổng số request	1	177	132
Thông lượng (req/giây)	0,40	0,75	0,70
Tỉ lệ thành công	100%	95,0%	87,0%
Số lỗi (rớt kết nối EOF)	0	9	17
Tỉ lệ lỗi HTTP	0%	5,0%	13,0%

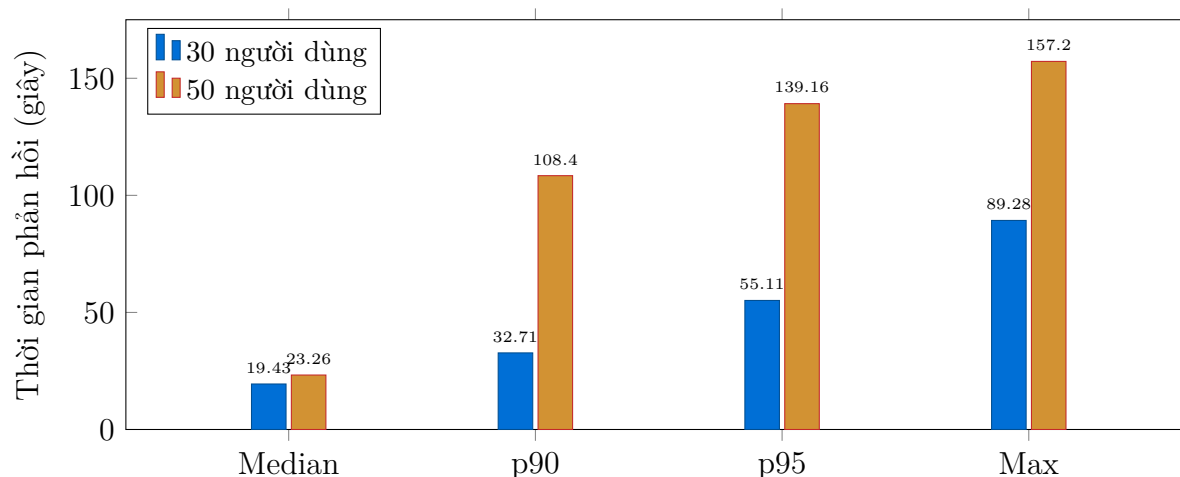
3.2. Phân bố thời gian phản hồi AI

Bảng 4: Phân bố thời gian phản hồi AI theo mức tải (đơn vị: giây)

Thời gian phản hồi	Baseline	30 user	50 user
Nhanh nhất (min)	–	0,59	1,34
Trung vị (median)	2,5	19,43	23,26
Trung bình (avg)	2,5	20,90	36,28
p90	–	32,71	108,40
p95	–	55,11	139,16
Chậm nhất (max)	2,5	89,28	157,20

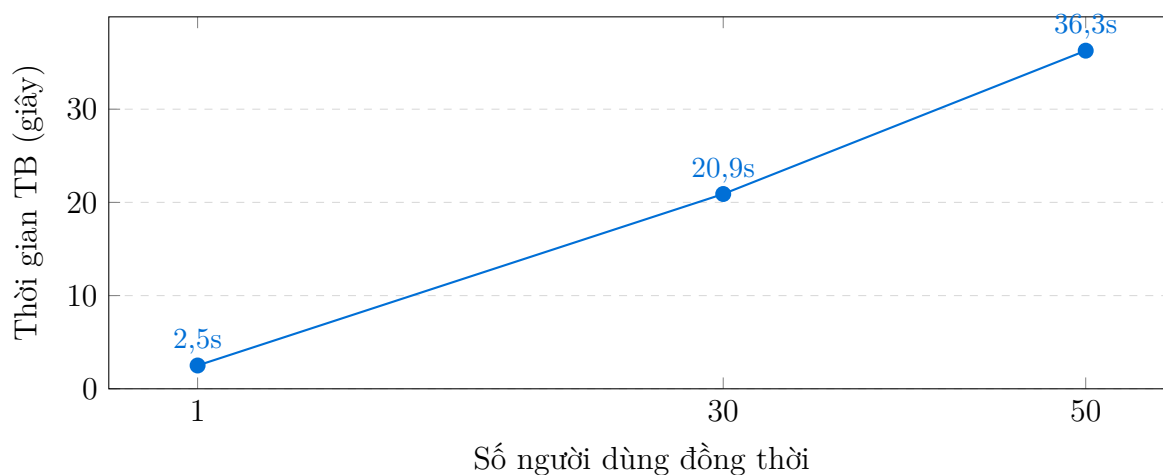
3.3. Biểu đồ trực quan

Hình 1 so sánh phân bố thời gian phản hồi giữa hai mức tải. Có thể thấy độ trễ tăng rất mạnh ở các percentile cao khi chuyển từ 30 lên 50 người dùng: p95 nhảy từ 55 giây lên 139 giây.



Hình 1: So sánh phân bố thời gian phản hồi AI ở mức 30 và 50 người dùng đồng thời.

Hình 2 cho thấy đặc trưng bão hoà: thời gian phản hồi trung bình tăng gần tuyến tính theo số người dùng, trong khi thông lượng không tăng — dấu hiệu điển hình của một hệ thống đã chạm trần xử lý.



Hình 2: Thời gian phản hồi trung bình tăng gần tuyến tính theo số người dùng (baseline → 30 → 50), trong khi thông lượng giữ nguyên $\approx 0,7$ req/giây.

3.4. Diễn biến theo mức tải

Bảng 5: Hành vi hệ thống quan sát theo từng giai đoạn tải

Giai đoạn	Số user	Hành vi hệ thống
Tải thấp	5	Ổn định, phản hồi nhanh, không có lỗi
Tải trung	15	Độ trễ tăng rõ, xuất hiện lỗi rút kết nối đầu tiên
Tải cao	30	Thông lượng chững lại, nhiều lỗi EOF, tỉ lệ lỗi 5%
Quá tải	50	Bão hoà hoàn toàn, p95 tới 139 giây, tỉ lệ lỗi 13%

4. Phân tích kết quả

4.1. Thông lượng đạt trần

Thông lượng gần như không đổi giữa 30 và 50 người dùng (0,75 so với 0,70 request/giây). Điều này chứng minh hệ thống đã **đạt trần xử lý** ở mức $\approx 0,7-0,75$ request/giây. Việc tăng thêm người dùng không làm tăng năng lực xử lý mà chỉ kéo dài hàng đợi, khiến độ trễ và tỉ lệ lỗi tăng vọt.

4.2. Giải thích bằng định luật Little

Định luật Little trong lý thuyết hàng đợi phát biểu:

$$L = \lambda \times W$$

trong đó L là số yêu cầu trung bình đang trong hệ thống (mức đồng thời), λ là thông lượng (request/giây) và W là thời gian phản hồi trung bình. Khi hệ thống đã bão hoà, thông lượng λ bị chặn trên bởi năng lực xử lý:

$$\lambda_{\max} \approx \frac{N_{\text{song song}}}{W_{\text{phục vụ}}}$$

Áp dụng cho đợt đo thứ nhất ở mức 50 người dùng: $\lambda = 0,70$ req/giây và $W = 36,28$ giây cho $L \approx 25$ — tức trung bình chỉ khoảng 25 trong số 50 người dùng thực sự có yêu cầu đang được hệ thống giữ, phần còn lại đã bị huỷ do lỗi. Với thời gian phục vụ cơ sở $W_{\text{phục vụ}} \approx 2,5$ giây đo ở baseline, mức song song hiệu dụng chỉ đạt $N_{\text{song song}} \approx 0,70 \times 2,5 \approx 1,8$.

Con số $\approx 1,8$ này ban đầu được quy cho giới hạn song song của Ollama. Tuy nhiên rà soát mã nguồn (Mục 5) cho thấy nó phản ánh một **điểm tuần tự hoá ở tầng ứng dụng**: máy chủ chỉ có một vòng lặp sự kiện và vòng lặp này bị các lời gọi đồng bộ chiếm dụng, nên dù có bao nhiêu người dùng thì mức song song hiệu dụng vẫn xấp xỉ 1. Đây là lý do sau khi khắc phục, thông lượng tăng lên 1,10 req/giây mà **không cần thay đổi gì về phần cứng**.

4.3. Hai nút thắt tách biệt

Đợt phân tích sau kiểm thử xác định **hai** nút thắt độc lập, trước đây bị gộp làm một:

1. **Tuần tự hoá ở tầng ứng dụng (đã khắc phục)**. Thư viện `psycopg2` và thư viện `ollama` đều đồng bộ nhưng được gọi trực tiếp trong hàm `async`. Mỗi lần chờ mạng tới Neon hoặc chờ mô hình sinh chữ, cả tiến trình dừng lại, không phục vụ được người dùng nào khác. Thêm vào đó, mỗi truy vấn mở một kết nối PostgreSQL mới, tốn trọn một lượt bắt tay TLS tới Neon (Singapore) trước khi câu lệnh được chạy.
2. **Tốc độ suy luận trên CPU (chưa khắc phục được bằng mã nguồn)**. Đo trực tiếp: ≈ 18 token/giây. Khoảng 1/6 số request cần gọi mô hình, mỗi lần tốn $\approx 5,5$ giây CPU. Trong 190 giây kiểm thử có ≈ 34 lượt gọi như vậy, tổng cộng ≈ 187 giây CPU — **bằng đúng thời lượng bài kiểm thử**. Nói cách khác CPU bị chiếm dụng 100% bởi suy luận, và chính điều này kéo chậm cả những request vốn chỉ chạm cơ sở dữ liệu.

4.4. Lỗi rớt kết nối

Toàn bộ lỗi ở đợt đo thứ nhất thuộc dạng EOF, tỉ lệ tăng từ 5% (30 người dùng) lên 13% (50 người dùng). Nguyên nhân trực tiếp là nút thắt số 1 ở trên: khi vòng lặp sự kiện bị chiếm dụng, request xếp hàng vượt quá timeout 180 giây và bị hủy. Sau khi khắc phục, **tỉ lệ lỗi về 0%** ở cùng mức tải — xác nhận lỗi đến từ tầng ứng dụng chứ không phải từ giới hạn kết nối của Neon.

Nhận định trọng yếu

Kết luận ban đầu “nút thắt duy nhất là mô hình ngôn ngữ” là **chưa đầy đủ**. Phần lớn lỗi và phần lớn độ trễ đuôi ở mức 50 người dùng đến từ cách tầng ứng dụng gọi các thư viện đồng bộ — sửa được bằng mã nguồn, chi phí bằng không. Sau khi gỡ lớp này, nút thắt thật sự mới lộ ra là tốc độ sinh token trên CPU, và đây là bài toán phần cứng.

5. Tối ưu hoá tầng ứng dụng và kết quả đo lại

5.1. Các thay đổi đã thực hiện

Bảng 6: Danh mục thay đổi mã nguồn nhằm gỡ nút thắt tầng ứng dụng

Tệp	Vấn đề	Cách khắc phục
postgres/ sql_runner.py	Mở kết nối mới cho mỗi truy vấn; psycopg2 đồng bộ chạy thẳng trong vòng lặp sự kiện	Dùng ThreadedConnectionPool (4–16 kết nối) và đẩy truy vấn sang ThreadPoolExecutor; thêm statement_timeout 20 giây; loại bỏ kết nối lỗi khỏi pool
ollama/llm.py	Vòng lặp đọc chuỗi token đồng bộ chiếm dụng vòng lặp sự kiện suốt thời gian sinh câu trả lời	send_request chuyển sang asyncio.to_thread; stream_request đọc token trên luồng riêng và chuyển về vòng lặp sự kiện qua hàng đợi
main.py	Cửa sổ ngữ cảnh 8.192 token quá dư; độ dài sinh không giới hạn	num_ctx = 4096, num_predict = 512; tham số pool đưa ra biến môi trường

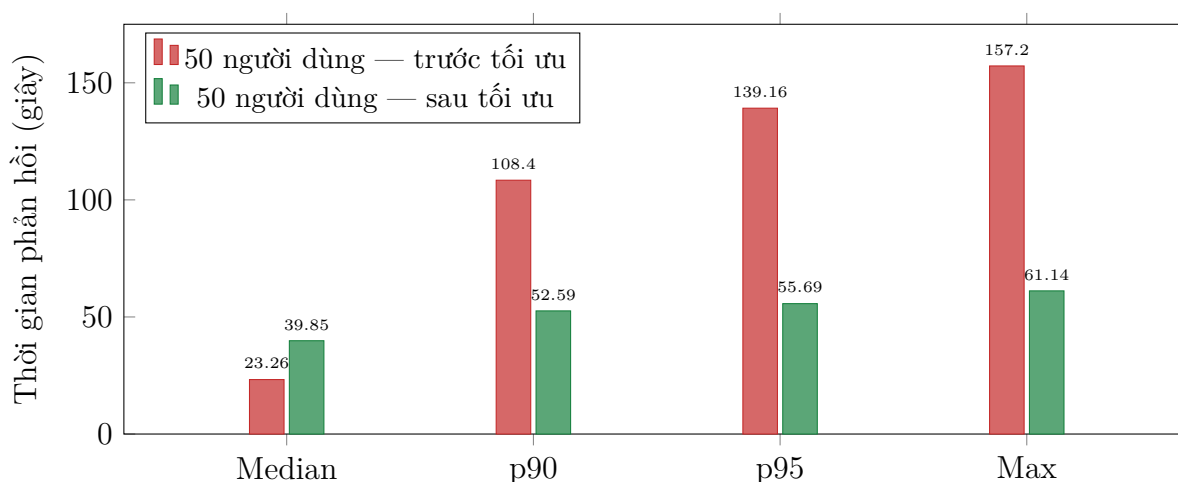
Toàn bộ thay đổi nằm ở **cách gọi** cơ sở dữ liệu và mô hình, không sửa logic nghiệp vụ, không sửa câu lệnh SQL và không đổi phần cứng.

5.2. Kết quả đo lại ở mức 50 người dùng

Kịch bản B được chạy lại nguyên vẹn (cùng bộ câu hỏi, cùng cấu hình ramping-vus, cùng máy). Kết quả trình bày ở Bảng 7.

Bảng 7: So sánh trước và sau tối ưu tại mức 50 người dùng đồng thời

Chỉ số	Trước tối ưu	Sau tối ưu	Thay đổi
Tổng số request	132	203	+54%
Thông lượng (req/giây)	0,70	1,10	+57%
Tỉ lệ thành công	87,1%	100%	+12,9 điểm
Số lỗi	17	0	-100%
Nhanh nhất (min), giây	1,34	0,40	-70%
Trung vị (median), giây	23,26	39,85	+71%
Trung bình (avg), giây	36,28	37,74	+4%
p90, giây	108,40	52,59	-51%
p95, giây	139,16	55,69	-60%
Chậm nhất (max), giây	157,20	61,14	-61%
1 người dùng (câu chạm CSDL)	≈ 4 giây	≈ 0,1 giây	-97%



Hình 3: Phân bố thời gian phản hồi tại mức 50 người dùng, trước và sau tối ưu. Đuôi trễ (p90, p95, max) giảm khoảng một nửa; trung vị tăng do hàng đợi trở nên công bằng hơn.

5.3. Diễn giải kết quả

Cần lưu ý hai chỉ số **không** cải thiện, và đây không phải hồi quy:

- **Trung bình gần như đứng yên** (36,28 → 37,74 giây). Tổng năng lực xử lý bị chặn bởi CPU, nên tối ưu tăng ứng dụng chỉ phân phối lại thời gian chờ chứ không tạo thêm năng lực suy luận.
- **Trung vị tăng** (23,26 → 39,85 giây). Trước tối ưu, một số request được phục vụ nhanh trong khi 17 request khác bị bỏ đói đến mức timeout và **không được tính vào thống kê độ trễ**. Sau tối ưu, hàng đợi công bằng: mọi request đều được phục vụ (0 lỗi) và cùng chịu mức chờ tương đương. Trung vị cao hơn ở đây là hệ quả của việc **không còn bỏ rơi request nào**, chứ không phải hệ thống chậm đi.

Chỉ số phản ánh đúng năng lực hệ thống là thông lượng: +57% với cùng phần cứng.

Đánh giá đợt tối ưu

Đợt tối ưu đạt mục tiêu về **độ tin cậy** (lỗi 13% → 0%) và **thông lượng** (+57%), nhưng chưa đạt mục tiêu về **độ trễ**: p95 55,7 giây vẫn cách rất xa ngưỡng SLA 10 giây. Phần còn lại phụ thuộc vào năng lực suy luận, tức phần cứng hoặc lựa chọn mô hình.

6. Thử nghiệm cấu hình tầng suy luận

Sau đợt tối ưu mã nguồn, bốn phương án cấu hình tầng suy luận được thử nghiệm để xác định xem có thể gỡ tiếp nút thắt mà không cần đầu tư phần cứng hay không. Mọi thử nghiệm dùng chung kịch bản, bộ câu hỏi và mã nguồn đã tối ưu; chỉ khác cấu hình Ollama hoặc mô hình.

6.1. Số lượng luồng suy luận song song

Rà soát nhật ký Ollama phát hiện tham số mặc định trên máy kiểm thử là `OLLAMA_NUM_PARALLEL: 1` — nghĩa là máy chủ mô hình chỉ xử lý **một** yêu cầu tại một thời điểm, mọi yêu cầu còn lại xếp hàng. Đây là một tham số cấu hình, không phải giới hạn phần cứng.

Bảng 8: So sánh các cấu hình tầng suy luận tại mức 50 người dùng đồng thời

Cấu hình	Request	RPS	Trung vị	p90	p95	Max
NP=1 (CPU)	203	1,10	39,85	52,59	55,69	61,14
NP=4 (CPU)	360	2,07	19,48	26,64	46,97	61,31
NP=4 + iGPU	192	1,03	40,35	62,29	65,42	71,36
NP=8 (CPU)	169	0,97	36,48	62,73	121,63	142,86

Đơn vị thời gian: giây. NP = `OLLAMA_NUM_PARALLEL`. Cả bốn cấu hình đều đạt tỉ lệ thành công 100%.

Nâng `NUM_PARALLEL` từ 1 lên 4 làm **tăng gấp đôi thông lượng** (1,10 → 2,07 req/giây) và giảm trung vị 51%. Cơ chế ở đây là gộp lô liên tục (continuous batching): sinh token bị chặn bởi băng thông bộ nhớ chứ không phải bởi phép tính, nên xử lý bốn luồng cùng lúc chia sẻ được chi phí đọc trọng số mô hình. Đây là lý do batching vẫn có hiệu quả ngay cả khi chạy hoàn toàn trên CPU.

6.2. Ba phương án bị bác bỏ bằng số liệu

- **Dùng GPU tích hợp (Radeon 780M).** Ollama mặc định bỏ qua GPU tích hợp; bật bằng `OLLAMA_IGPU_ENABLE=1` thì ollama ps xác nhận 100% GPU. Tuy nhiên thông lượng **giảm 47%** so với CPU. Nguyên nhân: iGPU dùng chung băng thông bộ nhớ hệ thống với CPU, nên khi gộp lô nó thua 8 nhân CPU. Khuyến nghị “bật tăng tốc ROCm/Vulkan cho 780M” ở Mục 10 vì vậy **không còn giá trị** trên cấu hình này.
- **Tăng `NUM_PARALLEL` lên 8.** Bộ nhớ mô hình phình từ 4,1 GB lên 5,9 GB, 8 luồng tranh chấp 8 nhân, p95 vọt lên 121,63 giây. Giá trị 4 là điểm tối ưu trên phần cứng này.

- **Đổi sang mô hình nhỏ hơn (11ama3.2:1b).** Mỗi lượt gọi mô hình mất **23–27 giây** thay vì 2–4 giây của bản 3B. Mô hình 1B tuân thủ chỉ thị kém hơn nên sinh câu trả lời dài dòng và lặp vòng gọi công cụ, khiến tổng số token phải sinh tăng vọt. **Nhỏ hơn không đồng nghĩa với nhanh hơn** khi bài toán phụ thuộc vào khả năng gọi công cụ đúng.

6.3. Kết quả với cấu hình tốt nhất

Bảng 9: Kết quả cuối cùng: mã nguồn đã tối ưu và NUM_PARALLEL=4

Chỉ số	30 user (ban đầu)	30 user (cuối cùng)	50 user (ban đầu)	50 user (cuối cùng)
Tổng số request	177	357	132	360
Thông lượng (req/giây)	0,75	1,61	0,70	2,07
Tỉ lệ thành công	95,0%	100%	87,1%	100%
Số lỗi	9	0	17	0
Trung vị, giây	19,43	10,03	23,26	19,48
p90, giây	32,71	18,68	108,40	26,64
p95, giây	55,11	23,96	139,16	46,97
Max, giây	89,28	27,79	157,20	61,31

So với hiện trạng ban đầu, thông lượng tăng **115%** ở mức 30 người dùng và **196%** ở mức 50 người dùng, tỉ lệ lỗi về 0% ở cả hai mức — toàn bộ đạt được bằng sửa mã nguồn và đổi một tham số cấu hình, **không tốn chi phí phần cứng**.

6.4. Vì sao vẫn không đạt p95 dưới 10 giây

Có thể tính trực tiếp yêu cầu cần thiết. Mỗi lượt gọi mô hình sinh ≈ 120 token. Ở cấu hình tốt nhất, thông lượng suy luận tổng đo được là:

$$2,07 \text{ req/giây} \times \frac{1}{6} \times 120 \text{ token} \approx 41 \text{ token/giây}$$

Để đạt p95 dưới 10 giây ở mức 50 người dùng, theo định luật Little cần $\lambda \approx 50/5 = 10$ req/giây, tương đương:

$$10 \times \frac{1}{6} \times 120 \approx 200 \text{ token/giây}$$

Tức cần **gấp gần 5 lần** năng lực suy luận hiện có (ở mức 30 người dùng vẫn cần gấp $\approx 2,4$ lần). Cả ba phương án không tốn chi phí đều đã được thử và đều không đạt, nên khoảng cách này **không thể lấp bằng cấu hình hay mã nguồn**.

Sức chứa với ngưỡng p95 dưới 10 giây

Suy từ thông lượng bão hoà 1,61 req/giây ở mức 30 người dùng, cấu hình cuối cùng đáp ứng ngưỡng p95 dưới 10 giây ở khoảng **8–12 người dùng đồng thời** (trước tối ưu là 5–10). Muốn phục vụ 30–50 người dùng trong ngưỡng này thì bắt buộc phải đầu tư GPU rời hoặc chuyển sang API LLM đám mây.

7. Đánh giá theo tiêu chí nghiệm thu (SLA)

Bảng 10 đối chiếu kết quả đo với các ngưỡng chất lượng dịch vụ đề xuất cho một hệ thống hỏi đáp tương tác. Với cấu hình cuối cùng, hai tiêu chí về **độ tin cậy đã đạt** ở cả mức 30 và 50 người dùng; tiêu chí về **độ trễ vẫn không đạt** do giới hạn tốc độ suy luận trên CPU (xem Mục 6.4).

Bảng 10: Đối chiếu kết quả với tiêu chí nghiệm thu đề xuất

Tiêu chí	Ngưỡng mục tiêu	30 user (ban đầu)	30 user (cuối cùng)	50 user (cuối cùng)
Độ trễ p95	< 10 giây	Không đạt (55,11 giây)	Không đạt (23,96 giây)	Không đạt (46,97 giây)
Tỉ lệ thành công	≥ 99%	Không đạt	Đạt (100%)	Đạt (100%)
Tỉ lệ lỗi	≤ 1%	Không đạt	Đạt (0%)	Đạt (0%)

Sức chứa an toàn ước tính

Nếu tiêu chí bắt buộc là **không mất request**, cấu hình cuối cùng đã phục vụ được **50 người dùng đồng thời** với tỉ lệ thành công 100%. Nếu tiêu chí bao gồm cả **p95 dưới 10 giây**, sức chứa chỉ ở mức **8–12 người dùng đồng thời** (trước tối ưu là 5–10), và muốn nâng lên thì bắt buộc phải đầu tư vào tăng suy luận theo các khuyến nghị ở Mục 10.

8. Giới hạn của kiểm thử

Kết quả trong báo cáo cần được diễn giải trong phạm vi các giới hạn sau:

- **Mô hình một máy:** ứng dụng, mô hình Ollama và công cụ k6 chạy trên cùng một máy, nên có sự cạnh tranh tài nguyên CPU giữa bên tạo tải và bên xử lý.
- **Phần cứng tiêu dùng:** máy dùng GPU tích hợp, không đại diện cho cấu hình máy chủ production có GPU rời; kết quả tuyệt đối sẽ khác trên hạ tầng mạnh hơn.
- **Câu hỏi đơn giản:** bộ câu hỏi thiên về đếm/tổng hợp; câu hỏi phức tạp (nhiều join, cửa sổ thời gian) sẽ làm tăng cả chi phí SQL lẫn suy luận.
- **Độ dài kiểm thử:** mỗi kịch bản kéo dài vài phút, chưa đánh giá độ ổn định dài hạn (rò rỉ bộ nhớ, suy giảm hiệu năng theo thời gian).
- **Cơ sở dữ liệu đám mây:** độ trễ mạng tới Neon có thể dao động theo thời điểm, ảnh hưởng nhỏ đến kết quả.

9. Kết luận

Sau tối ưu, hệ thống phục vụ được 50 người dùng đồng thời không mất request, nhưng độ trễ vẫn bị chặn bởi phần cứng.

Tối ưu mã nguồn cộng với việc đặt `OLLAMA_NUM_PARALLEL=4` đưa tỉ lệ lỗi từ 13% về **0%**, nâng thông lượng từ 0,70 lên **2,07 request/giây** (+196%) và giảm p95 từ 139 giây xuống **47,0 giây** (-66%) — toàn bộ trên cùng phần cứng, không phát sinh chi phí. Tuy nhiên p95 vẫn vượt xa ngưỡng SLA 10 giây.

Bài học rút ra: kết luận ban đầu quy toàn bộ nút thắt cho mô hình ngôn ngữ là chưa đầy đủ. Một phần đáng kể độ trễ và **toàn bộ lỗi** thực ra đến từ tầng ứng dụng — cụ thể là việc gọi thư viện đồng bộ trong vòng lặp sự kiện bất đồng bộ và việc không dùng connection pool. Đây là những lỗi sửa được bằng mã nguồn với chi phí bằng không, và nên được rà soát trước khi kết luận về nhu cầu phần cứng.

Sau khi lớp này được gỡ bỏ, nút thắt còn lại là năng lực suy luận thật sự. Một phần của nút thắt này cũng chỉ là cấu hình: `OLLAMA_NUM_PARALLEL` để mặc định bằng 1 khiến máy chủ mô hình xử lý tuần tự, và nâng lên 4 làm thông lượng tăng gần gấp đôi. Nhưng ba phương án còn lại không tốn chi phí — GPU tích hợp, `NUM_PARALLEL=8` và mô hình 1B — đều đã thử và đều cho kết quả xấu hơn (Mục 6).

Vì vậy khoảng cách còn lại tới ngưỡng p95 dưới 10 giây (cần gấp ≈ 5 lần năng lực suy luận ở mức 50 người dùng) **không thể lấp bằng cấu hình hay mã nguồn**. Để phục vụ nhiều người dùng hơn với độ trễ chấp nhận được, bắt buộc phải đầu tư vào tầng suy luận theo các khuyến nghị dưới đây.

10. Khuyến nghị cải thiện

10.1. Đã thực hiện trong đợt tối ưu này

Bảng 11: Các hạng mục đã hoàn thành và kết quả thực đo

Trạng thái	Giải pháp	Kết quả thực đo
Hoàn thành	Đưa lời gọi <code>psycopg2</code> và <code>ollama</code> ra khỏi vòng lặp sự kiện (thread pool và hàng đợi)	Tỉ lệ lỗi 13% $\rightarrow$ 0%; thông lượng +57%
Hoàn thành	Connection pool tới Neon (4–16 kết nối) thay cho mở kết nối mỗi truy vấn	Truy vấn chạm CSDL: $\approx$ 4 giây $\rightarrow$ $\approx$ 0,1 giây
Hoàn thành	Giới hạn <code>statement_timeout</code> 20 giây và <code>num_predict</code> 512 token	Chặn đuôi trễ: max 157 $\rightarrow$ 61 giây
Hoàn thành	Đặt <code>OLLAMA_NUM_PARALLEL=4</code> thay cho mặc định 1	Thông lượng $\times 1,9$; trung vị -51%

10.2. Đã thử và bị bác bỏ

Ba phương án dưới đây từng nằm trong danh mục khuyến nghị nhưng đã được thử nghiệm và **đo ra kết quả xấu hơn** (chi tiết ở Mục 6). Ghi lại tại đây để tránh lặp lại.

Bảng 12: Các phương án đã thử nghiệm và bị loại

Kết luận	Phương án	Kết quả đo
Loại	Bật tăng tốc GPU tích hợp Radeon 780M (OLLAMA_IGPU_ENABLE)	Thông lượng -47% ; iGPU chia sẻ băng thông RAM với CPU
Loại	Nâng OLLAMA_NUM_PARALLEL lên 8	p95 vọt lên 121,63 giây; 4 là điểm tối ưu
Loại	Đổi sang mô hình nhỏ hơn llama3.2:1b	Mỗi lượt gọi 23–27 giây thay vì 2–4 giây

10.3. Còn lại — cần đầu tư

Bảng 13: Các khuyến nghị nâng cao khả năng chịu tải

Ưu tiên	Giải pháp	Kỳ vọng
Cao	Trang bị GPU rời cho máy chủ suy luận	Tốc độ sinh token nhanh 10–20 lần — đòn bẩy duy nhất đủ lớn để đạt p95 dưới 10 giây
Cao	Hoặc chuyển sang API LLM đám mây (Anthropic/OpenAI) cho tải cao	Bỏ hoàn toàn trần CPU cục bộ; đổi lại phát sinh chi phí theo token
Trung bình	Cache câu hỏi và câu lệnh SQL lặp lại	Giảm số lần gọi mô hình. Lưu ý: bộ câu hỏi kiểm thử chỉ có 6 câu lặp lại nên cache sẽ cho tỉ lệ trùng gần 100% và kết quả đo sẽ không phản ánh tải thật; cần đánh giá lại bằng bộ câu hỏi đa dạng hơn

Ưu tiên	Giải pháp	Kỳ vọng
Trung bình	Chạy Uvicorn nhiều worker (<code>--workers 4</code>) hoặc đặt sau Gunicorn	Tận dụng đa nhân cho tầng ứng dụng (lợi ích hạn chế khi CPU đã bị suy luận chiếm)
Thấp	Thêm hàng đợi và thông báo “đang xử lý” cho người dùng	Cải thiện trải nghiệm khi tải cao

Người thực hiện
(Đã lập báo cáo)

Lê Đức Anh

Phụ lục A. Kịch bản k6 (rút gọn)

Listing 2: Logic phát tải và đo lường của mỗi người dùng ảo

```
import http from 'k6/http';
import { check } from 'k6';
import { Trend, Rate, Counter } from 'k6/metrics';

const aiResponseTime = new Trend('ai_response_time', true);
const aiSuccessRate = new Rate('ai_success_rate');
const aiErrors = new Counter('ai_errors');

export default function () {
  // conversation_id duy nhất -> tránh khoa 409 theo hỏi thoại
  const uid = `lt-${__VU}-${__ITER}-${Date.now()}-${Math.random()}`;
  const question = QUESTIONS[Math.floor(Math.random() * QUESTIONS.length)];

  const res = http.post(ENDPOINT,
    JSON.stringify({ message: question, conversation_id: uid, request_id: uid }),
    { headers: { 'Content-Type': 'application/json' }, timeout: '180s' });

  aiResponseTime.add(res.timings.duration);
  const ok = res.status === 200 && String(res.body).includes('data: [DONE]');
  aiSuccessRate.add(ok);
  if (!ok) aiErrors.add(1);

  check(res, {
    'HTTP 200': (r) => r.status === 200,
    'stream hoàn tất': (r) => String(r.body).includes('data: [DONE]'),
  });
}
```

Mã nguồn đầy đủ và số liệu thô có trong thư mục loadtest/ của kho mã: kịch bản ai_load_test.js (30 user) và ai_load_test_50.js (50 user); kết quả k6_summary.json và k6_summary_50.json (hiện trạng ban đầu), k6_summary_50_optimized.json (sau tối ưu mã nguồn), k6_30_np4.json và k6_50_np4.json (cấu hình cuối cùng), k6_50_gpu.json và k6_50_np8.json (hai phương án bị loại).

Các thay đổi mã nguồn của đợt tối ưu nằm ở: src/vanna/integrations/postgres/sql_runner.py, src/vanna/integrations/ollama/llm.py và main.py.

Tài liệu tham khảo

- [1] Grafana Labs, *k6 — Load testing tool for engineering teams*, tài liệu công cụ kiểm thử tải mã nguồn mở, phiên bản 2.1.0. <https://k6.io/docs/>. Truy cập ngày 04/08/2026.
- [2] Vanna AI, *Vanna 2.0: Turn Questions into Data Insights*, GitHub repository [vanna-ai/vanna](https://github.com/vanna-ai/vanna), phiên bản mã nguồn tham chiếu 2.0.2. <https://github.com/vanna-ai/vanna>. Truy cập ngày 04/08/2026.
- [3] Ollama, *Ollama — Run large language models locally*, tài liệu cấu hình song song (OLLAMA_NUM_PARALLEL) và phục vụ mô hình. <https://github.com/ollama/ollama>. Truy cập ngày 04/08/2026.
- [4] Neon, *Connection pooling*, tài liệu kết nối PostgreSQL và pooled connection cho ứng dụng. <https://neon.com/docs/connect/connection-pooling>. Truy cập ngày 04/08/2026.